MLOps (CS4085)

Building an Airflow ETL Pipeline

Daily Currency Exchange Rates: Frankfurter API → MongoDB

Submitted by

Muhammad Fezaan Shamshad

Roll No. 23L-0729

Submitted to

Usman Anwer

12 November 2026

1. Objective

The goal of this assignment is to build an automated ETL (Extract, Transform, Load) pipeline with Apache Airflow, set up through the Astro CLI. Every day the pipeline fetches currency exchange rates from the free Frankfurter API, converts them into a fixed and consistent record format, and stores them in MongoDB. Mongo Express runs as a separate Docker service so the stored data can be browsed in a web interface.

The three tasks pass data to each other only through named Airflow XComs. All services run in Docker, share one Docker network, and MongoDB keeps its data on a persistent named volume.

2. Tools and Technologies

Component	Role in the project
Astro CLI 1.47.0	Creates and runs the local Airflow project in Docker
Apache Airflow 3	Orchestrates and schedules the ETL tasks (DAG runs @daily)
Frankfurter API (v2)	Free public source of daily exchange rates
MongoDB 7	Primary database for the transformed records
Mongo Express	Web-based administration UI for MongoDB
Docker / Docker Compose	Containers, shared network and persistent volume
Python (requests, pymongo)	API calls and database access inside tasks
GitHub / DagsHub	Version control and code submission

3. Pipeline Architecture

The pipeline is a single DAG, currency_exchange_etl, with three tasks that run in sequence:

Frankfurter API → extract_exchange_rates → transform_exchange_rates → load_to_mongodb → MongoDB → Mongo Express

Task	Pulls (named XCom)	Pushes (named XCom)
extract_exchange_rates	None (calls the API)	raw_exchange_rates, retrieved_at
transform_exchange_rates	raw_exchange_rates, retrieved_at	transformed_exchange_rates
load_to_mongodb	transformed_exchange_rates	load_summary

API call: `https://api.frankfurter.dev/v2/rates?base=EUR&quotes=USD,GBP,JPY`. The v2 API returns a list of objects with the fields date, base, quote and rate.

Transformed record format: each rate becomes one document with the fields required by the assignment:

```
{
  "base_currency": "EUR",
  "target_currency": "USD",
  "exchange_rate": 1.1134,
  "rate_date": "2026-11-12",
  "source": "Frankfurter API",
  "retrieved_at": "2026-11-12T14:56:26"
}
```


4. Environment Setup

The Astro CLI was installed on Windows with winget, and a new project was created with astro dev init. The pymongo package was added to requirements.txt, and the example DAG was removed. Running astro dev start builds the image and starts Airflow in Docker.

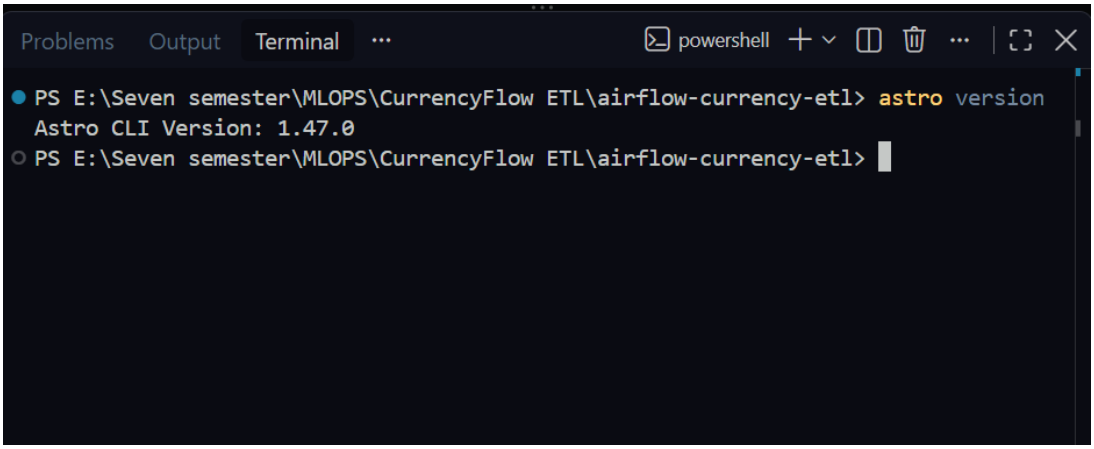


Figure 1: *Astro CLI installed and verified with astro version (v1.47.0).*

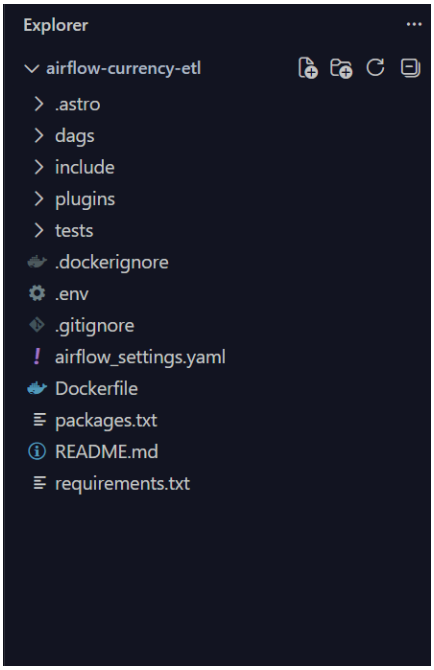


Figure 2: *Astro project initialized with astro dev init, showing the default project structure.*

```
PS E:\Seven semester\MLOPS\CurrencyFlow ETL\airflow-currency-etl> astro dev ps
  Name                                          State       Ports
  airflow-currency-etl_7b9477-api-server-1      running     13931
  airflow-currency-etl_7b9477-triggerer-1       running
  airflow-currency-etl_7b9477-dag-processor-1   running
  airflow-currency-etl_7b9477-scheduler-1       running
  airflow-currency-etl_7b9477-db-migration-1    exited
  airflow-currency-etl_7b9477-postgres-1        running     18737
PS E:\Seven semester\MLOPS\CurrencyFlow ETL\airflow-currency-etl>
```

Figure 3: *Airflow services (scheduler, API server, DAG processor, triggerer, Postgres metadata DB) running in Docker via Astro CLI.*

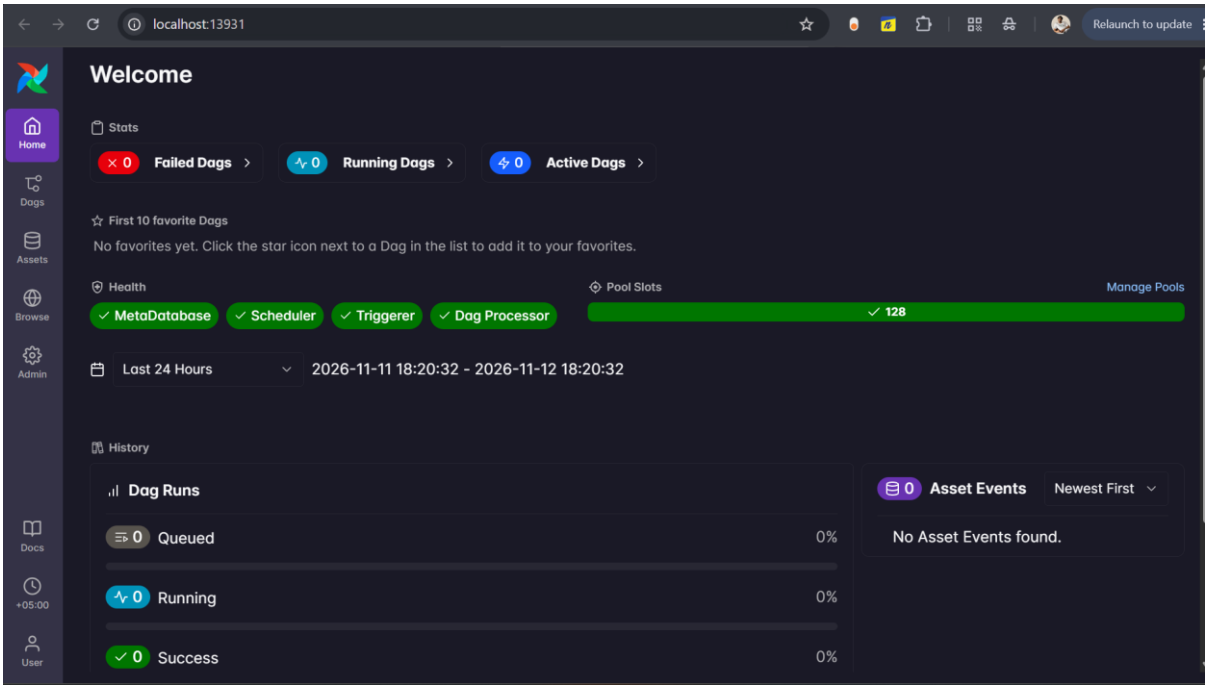


Figure 4: *Airflow 3 web UI running locally; MetaDatabase, Scheduler, Triggerer and DAG Processor all healthy.*

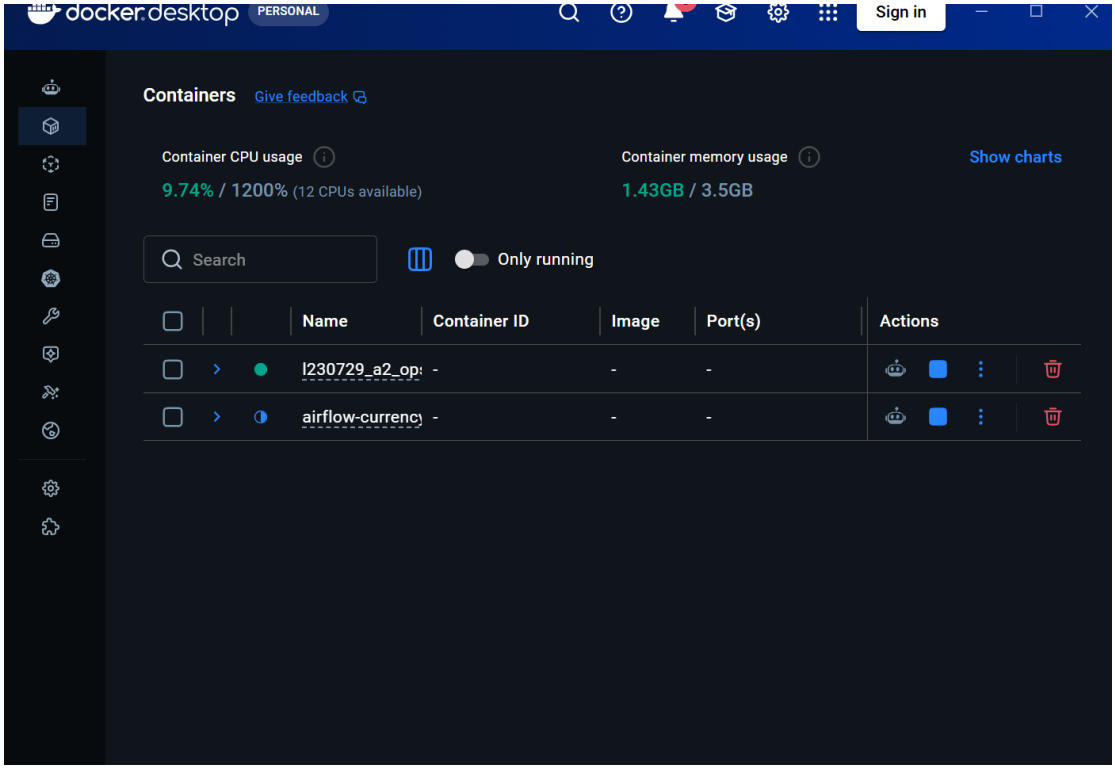


Figure 5: *Astro project containers as seen in Docker Desktop.*

5. MongoDB and Mongo Express Configuration

MongoDB and Mongo Express were added as extra Docker services in docker-compose.override.yml, which the Astro CLI merges with its own Compose file automatically.

5.1 Docker Networking

Both new services join the airflow bridge network that Astro creates for the project. Because every container is on the same network, the Airflow tasks reach MongoDB by its service name (mongodb:27017), and Mongo Express connects to MongoDB the same way. No container IP addresses are hard-coded.

5.2 Persistent Storage

MongoDB's data directory (/data/db) is mounted on the named volume mongo_data. The volume lives outside the container, so the stored exchange rates survive when containers are stopped, restarted or recreated.

5.3 Image Metadata (Labels)

Each service carries Docker labels (project, description, maintainer) that describe the container and make it easy to identify.

```
PS E:\Seven semester\MLOPS\CurrencyFlow ETL\airflow-currency-etl> astro dev ps
  Name                                          State       Ports
  airflow-currency-etl_7b9477-scheduler-1       running
  airflow-currency-etl_7b9477-api-server-1      running     8080
  airflow-currency-etl_7b9477-triggerer-1       running
  airflow-currency-etl_7b9477-dag-processor-1   running
  airflow-currency-etl_7b9477-db-migration-1    exited
  currency-mongo-express                        running     8081
  airflow-currency-etl_7b9477-postgres-1        running     5432
  currency-mongodb                              running     27017
PS E:\Seven semester\MLOPS\CurrencyFlow ETL\airflow-currency-etl>
```

Figure 6: *MongoDB (27017) and Mongo Express (8081) running alongside the Airflow services.*

```
PS E:\Seven semester\MLOPS\CurrencyFlow ETL\airflow-currency-etl> docker network
inspect airflow-currency-etl_7b9477_airflow --format "{{range .Containers}}{{.Nam
e}}  {{end}}"
airflow-currency-etl_7b9477-scheduler-1  airflow-currency-etl_7b9477-api-server-1
  airflow-currency-etl_7b9477-postgres-1  airflow-currency-etl_7b9477-dag-process
or-1  currency-mongo-express  currency-mongodb  airflow-currency-etl_7b9477-trigg
erer-1
PS E:\Seven semester\MLOPS\CurrencyFlow ETL\airflow-currency-etl>
```

Figure 7: *Docker networking: Airflow, MongoDB and Mongo Express containers attached to the same network.*

```
3807a25c0bb8   none                                          null      local
DRIVER    VOLUME NAME
local     840369d84bb5a44e66af2f9707ef915a6243dc745ddcade4a5d20ac1567b166b
local     airflow-currency-etl_7b9477_airflow_logs
local     airflow-currency-etl_7b9477_mongo_data
local     airflow-currency-etl_7b9477_postgres_data
local     l230729_a2_ops_pgdata
```

Figure 8: *Named Docker volume mongo_data providing persistent storage for MongoDB.*

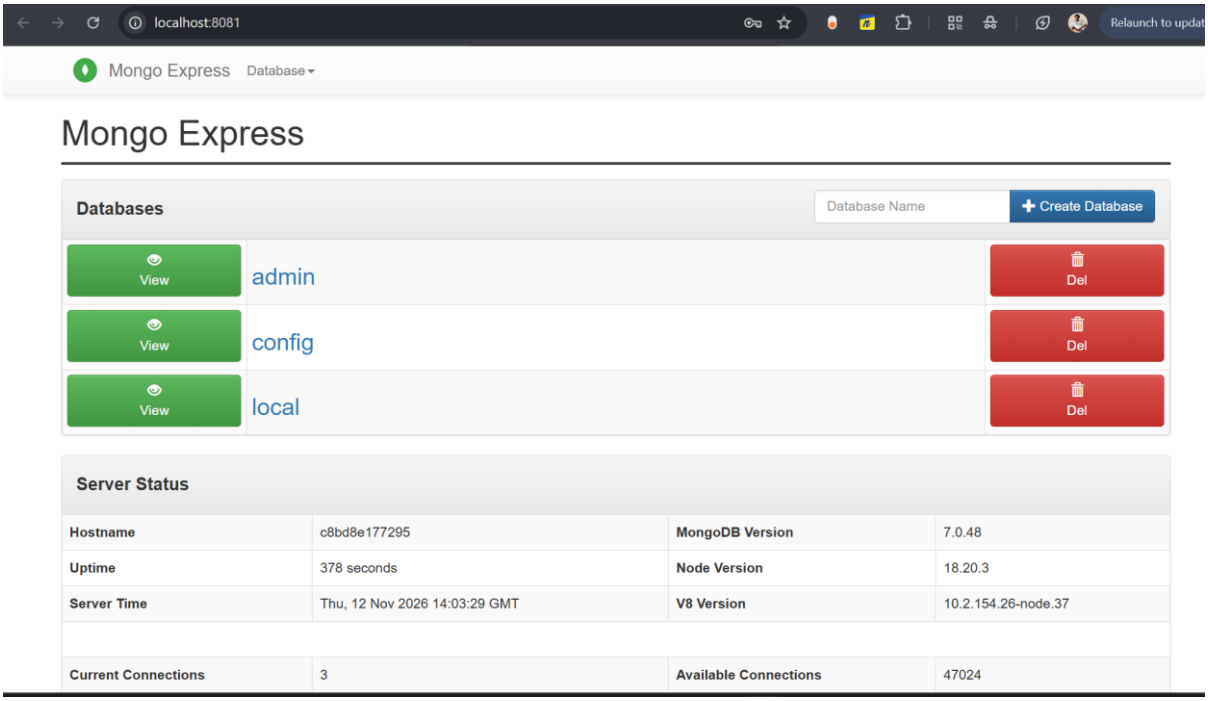


Figure 9: *Mongo Express web interface connected to MongoDB.*

6. DAG Implementation

The DAG is defined in `dags/currency_etl_dag.py`. It runs on the @daily schedule, retries a failed task twice, and uses the PythonOperator for all three tasks.

- **Extract:** calls the Frankfurter API for EUR to USD, GBP and JPY, records the retrieval time in UTC, and pushes the raw response and the timestamp as two named XComs.
- **Transform:** pulls both XComs by key and maps each API row (base, quote, rate, date) to the required fields, adding source and retrieved_at.
- **Load:** pulls the transformed records and writes them to currency_db.exchange_rates with an upsert on (base_currency, target_currency, rate_date), so re-running the same day updates records instead of duplicating them. A load summary is pushed as a final XCom.
- **Named XCom:** every transfer uses ti.xcom_push(key=...) and ti.xcom_pull(task_ids=..., key=...) with an explicit key, instead of the default return_value.

The MongoDB connection string is supplied through the Astro `.env` file as `MONGO_URI=mongodb://root:example@mongodb:27017/`.

7. Execution and Results

7.1 DAG Run

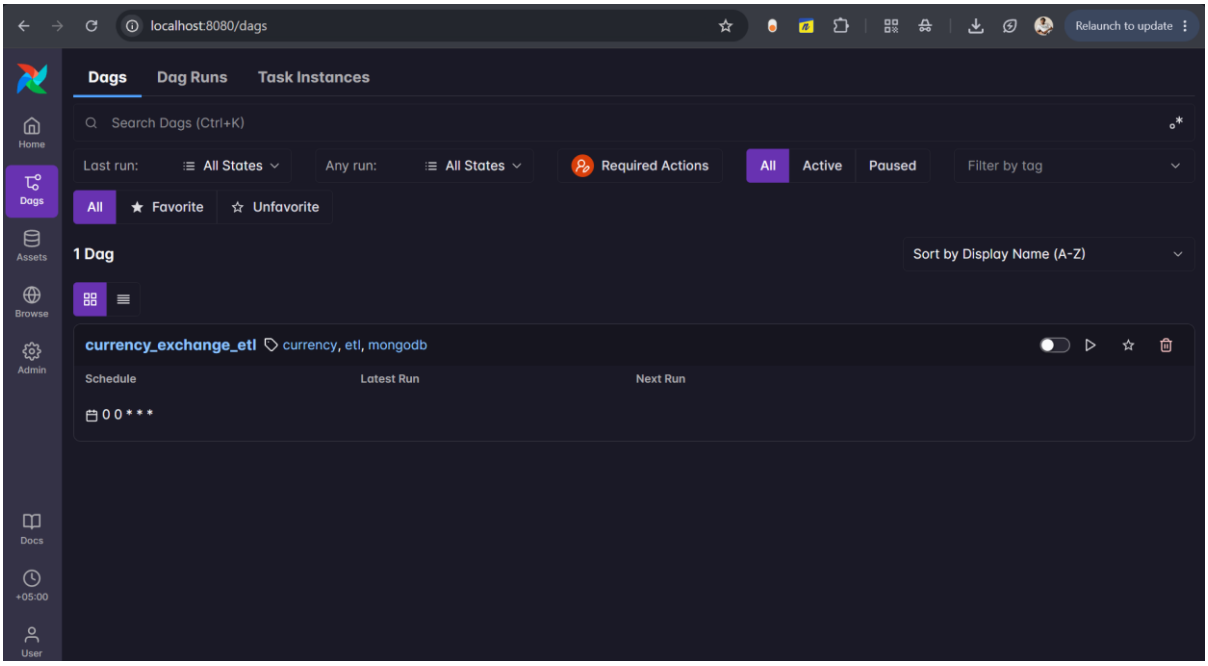


Figure 10: *DAG currency_exchange_etl detected by Airflow, scheduled @daily (cron 0 0 * * *).*

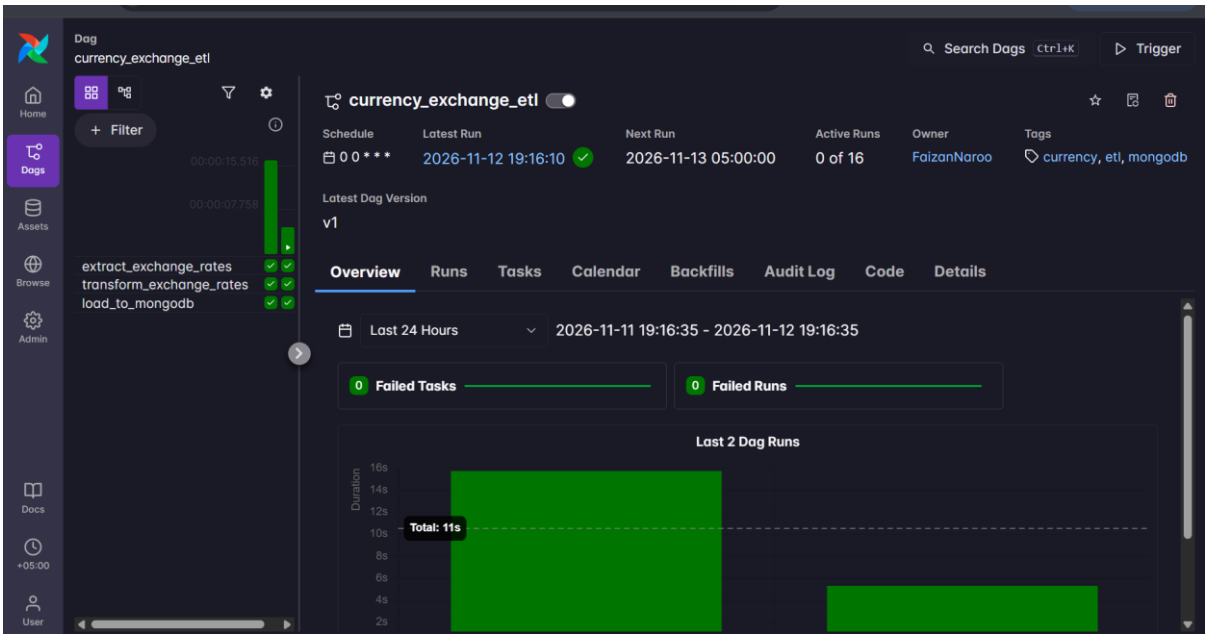


Figure 11: *Successful DAG runs: all three tasks (extract, transform, load) completed.*

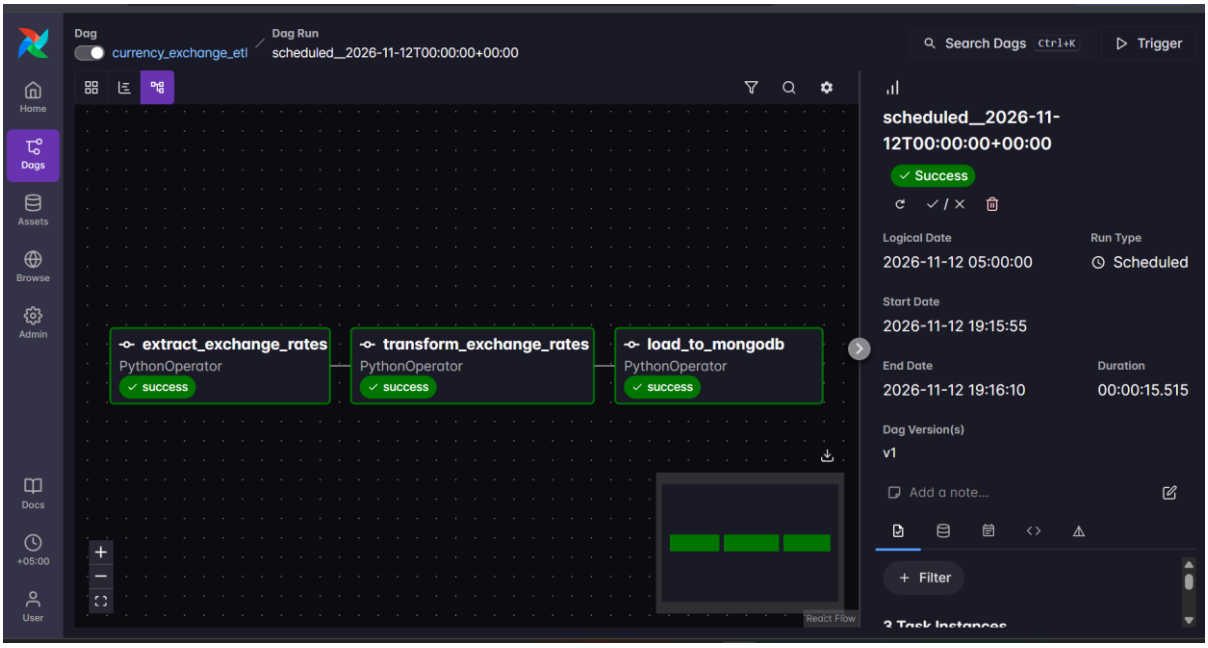


Figure 12: *Graph view of the successful run: extract_exchange_rates → transform_exchange_rates → load_to_mongodb.*

7.2 Task Logs and Named XComs

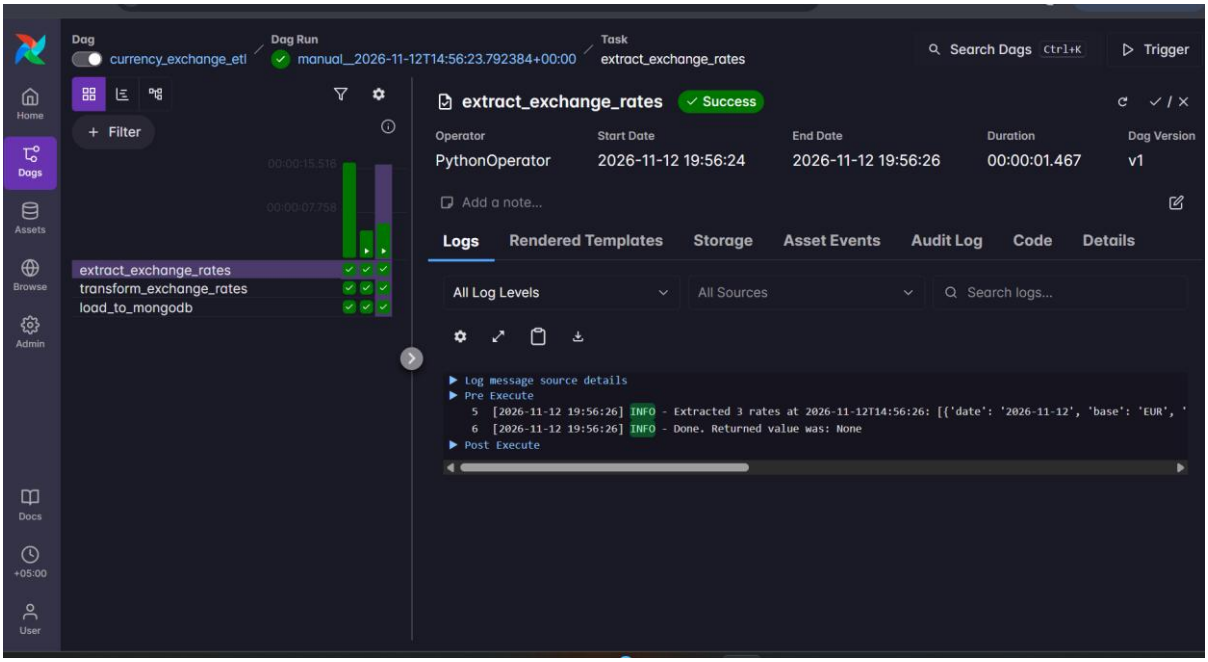


Figure 13: *Extract task log: 3 raw EUR rates fetched from the Frankfurter API.*

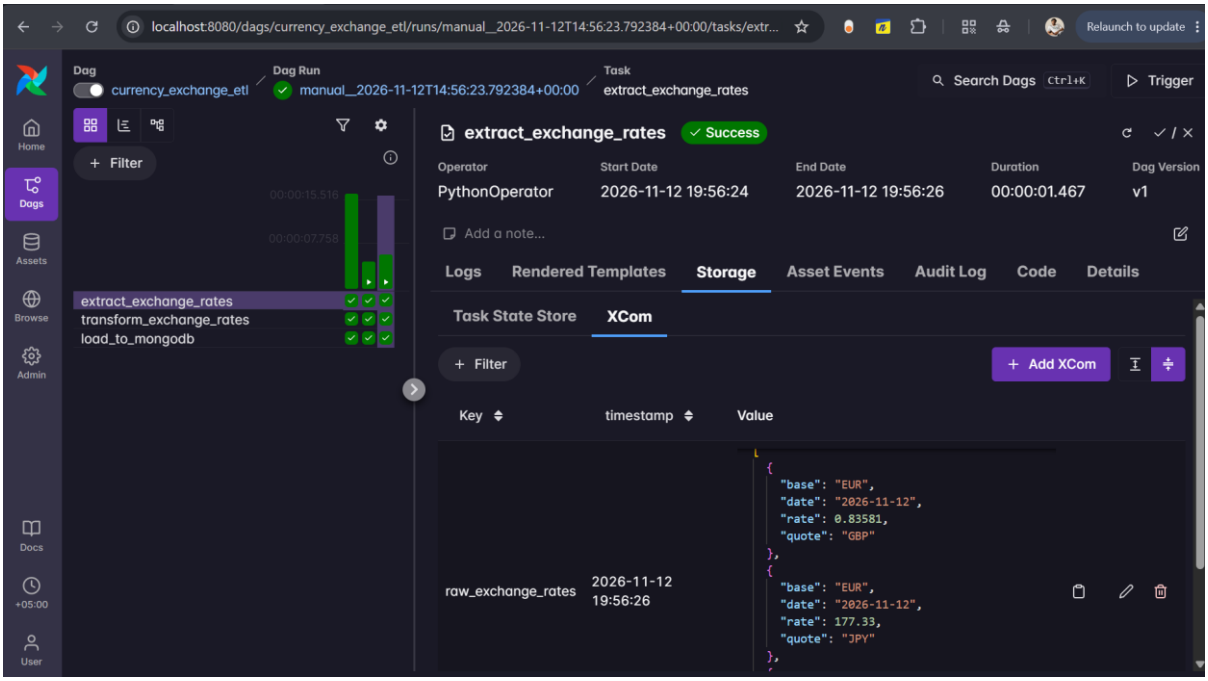


Figure 14: *Named XComs pushed by extract: raw_exchange_rates and retrieved_at.*

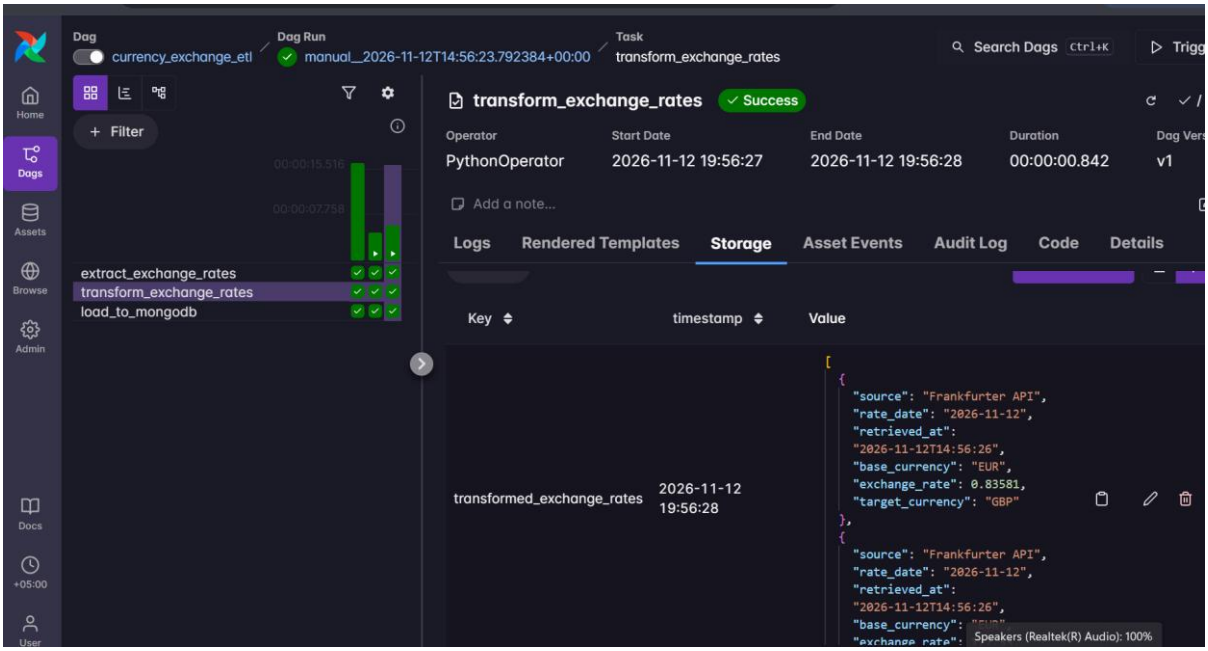


Figure 15: *Named XCom transformed_exchange_rates: records in the required format.*

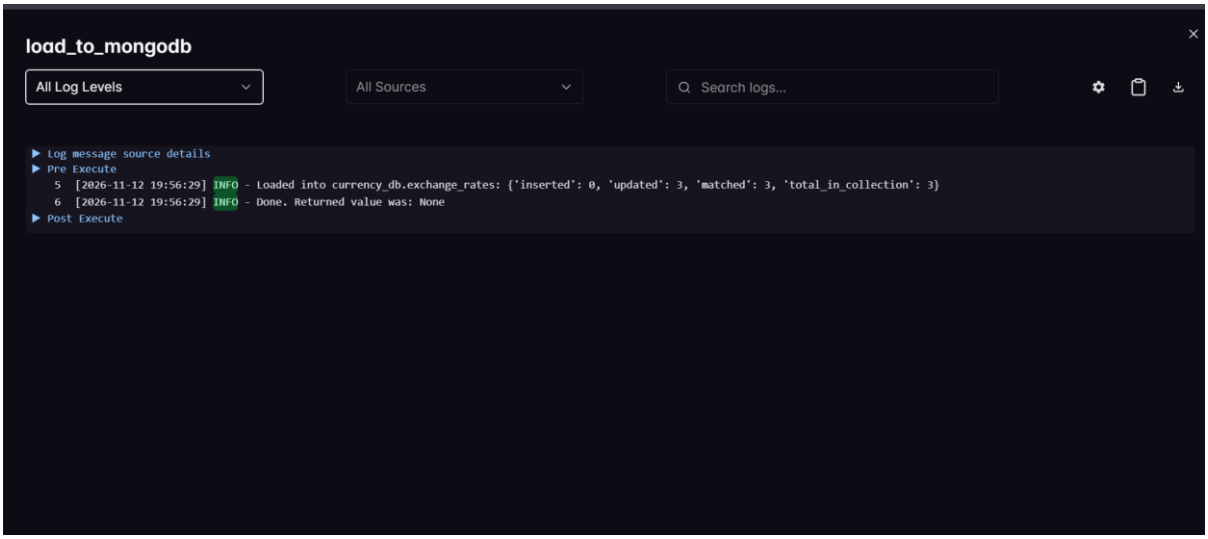


Figure 16: *Load task log: records written to currency_db.exchange_rates. On this re-run the upsert matched and updated the 3 existing records, so no duplicates were created.*

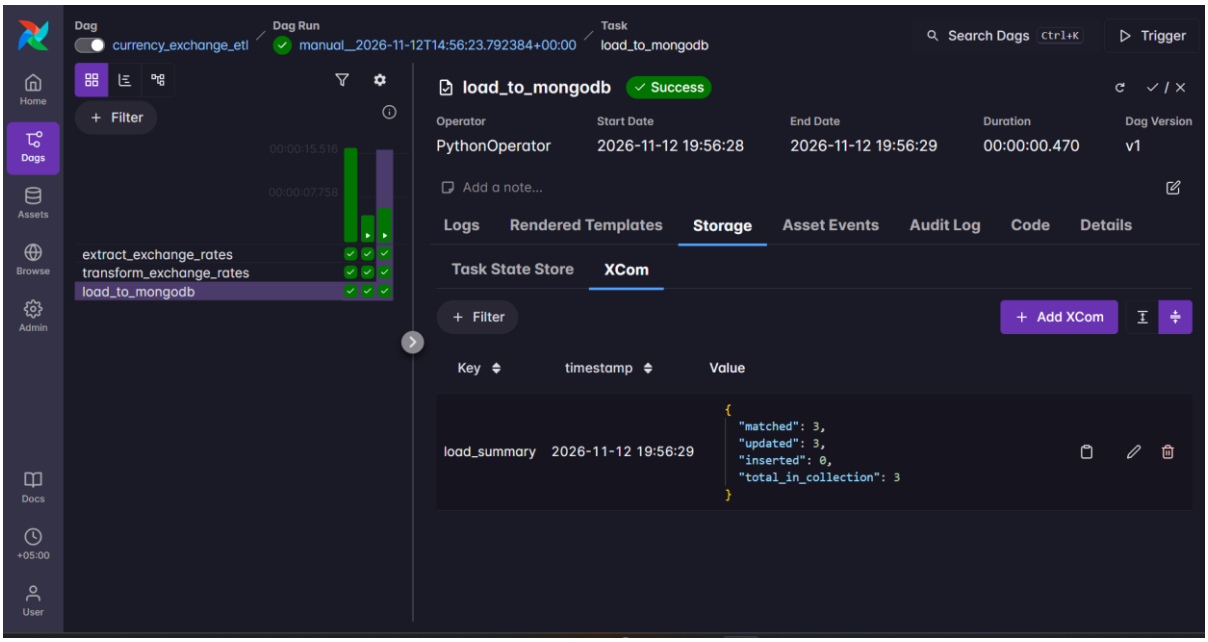


Figure 17: *Named XCom load_summary pushed by the load task.*

7.3 Data in MongoDB (Mongo Express)

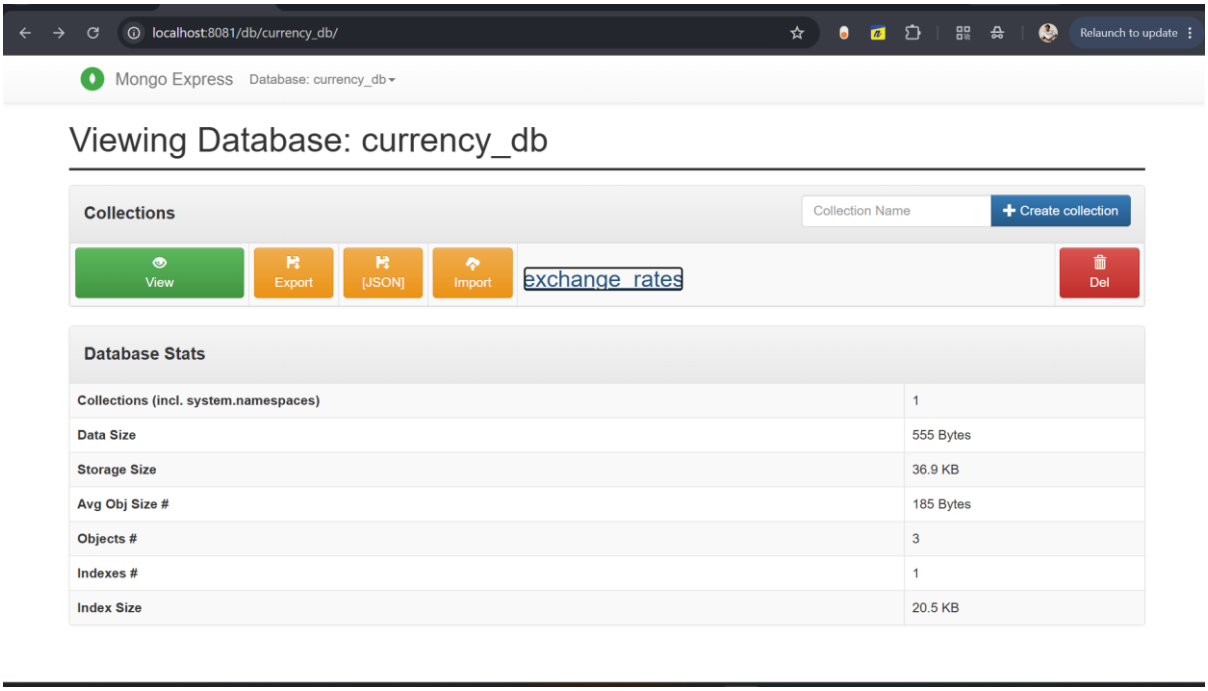


Figure 18: *Database currency_db with the exchange_rates collection, created by the load task.*

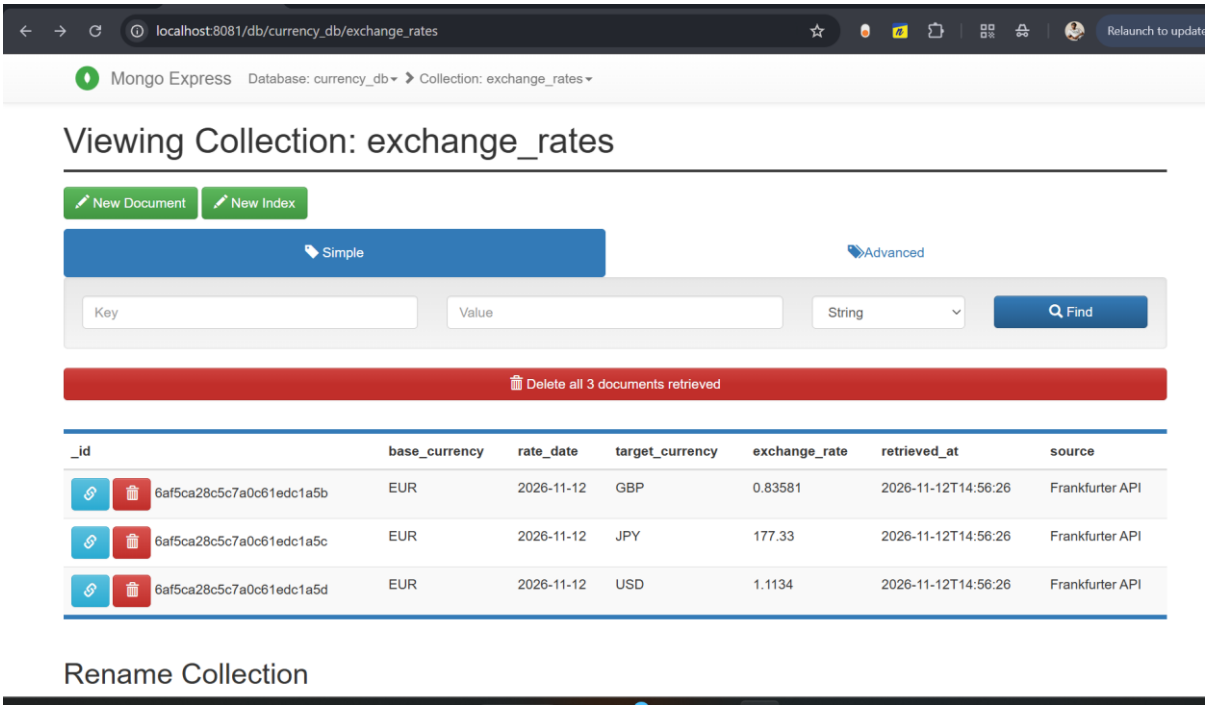


Figure 19: *The exchange_rates collection holding 3 transformed records (EUR to GBP, JPY and USD).*

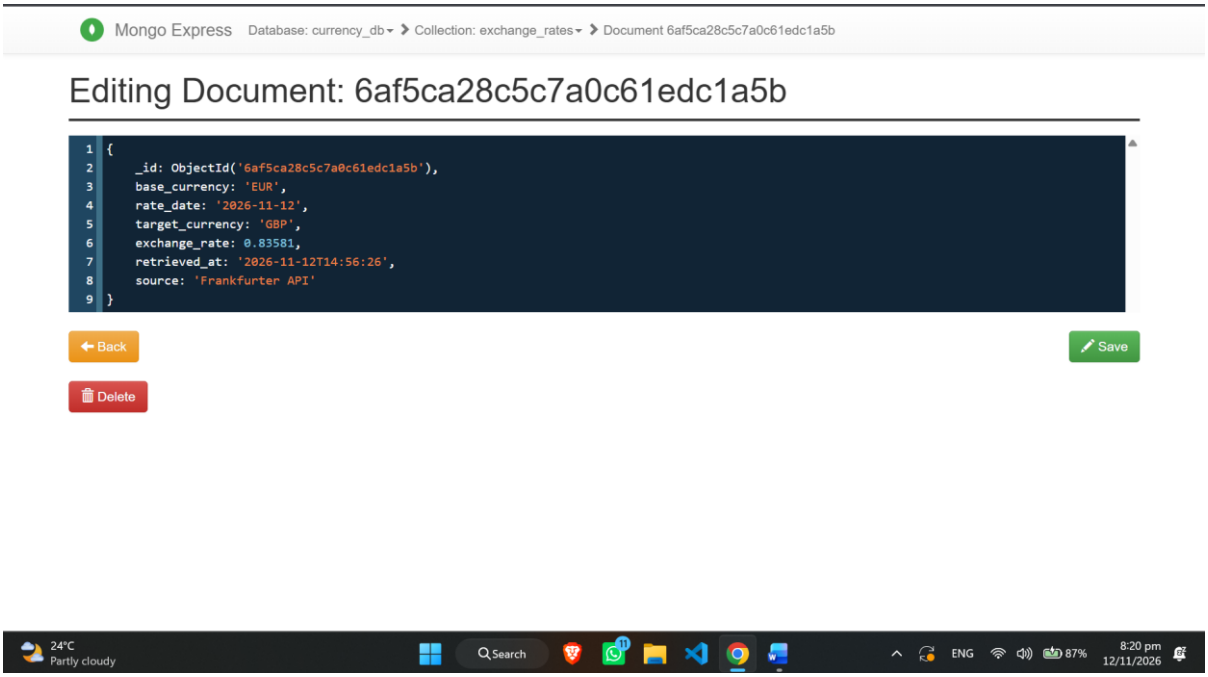


Figure 20: *A single stored document showing all six required fields.*

8. Source Code Repository

GitHub: https://github.com/FaizanNaroo/airflow_currency_etl

DagsHub: https://dagshub.com/FaizanNaroo/airflow_currency_etl

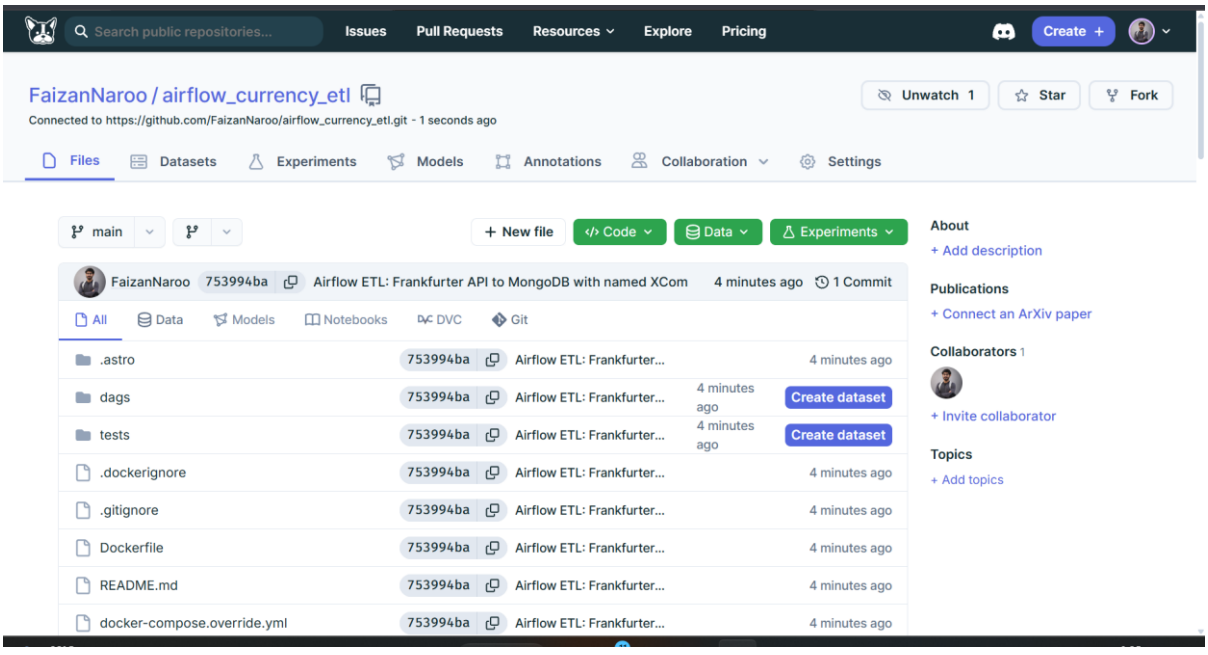


Figure 21: *DagsHub repository connected to the GitHub repository.*

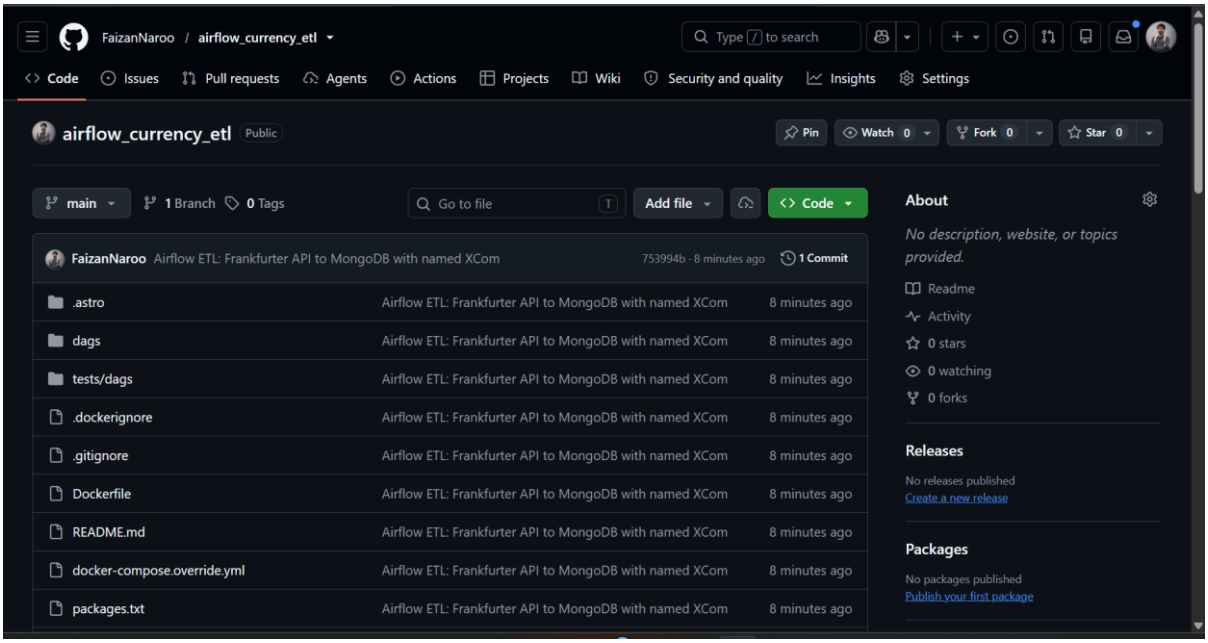


Figure 22: *GitHub repository containing the Astro project, DAG and Docker Compose override.*

9. Conclusion

A working daily ETL pipeline was built with Apache Airflow through the Astro CLI. It extracts EUR exchange rates from the Frankfurter API, transforms them into the required structure, and loads them into MongoDB, with all data passed between tasks through named XComs. MongoDB and Mongo Express run as Docker services on the same network as Airflow, and MongoDB's data persists on a named volume. The upsert-based load keeps the collection free of duplicates when a day is re-run, so the pipeline can run every day without manual work.